SpendWise

Introduction:

Project Title : SpendWise

Team ID :

Role : Coding and development

Team members :

- Archana S
- Keerthika A
- Keerthana M

Project Title:

SpendWise – A Simple Backend Learning Project using Node.js, Express, MongoDB, and JWT

Project Overview:

SpendWise is a backend application designed to help beginner developers learn how to build secure REST APIs using Node.js, Express, MongoDB, and JWT authentication. The project focuses on implementing user authentication and basic CRUD operations in a clean and simple way so students can understand how backend systems work in real-world scenarios.

Key Objectives:

- Provide secure user authentication using JWT
- Enable basic CRUD operations on user data (transactions/records)
- Demonstrate MongoDB integration using Mongoose

- Show how protected routes work using middleware
- Maintain a simple and beginner-friendly backend architecture

Target Users:

- Beginner backend developers
- Students learning Node.js and Express
- Freshers preparing for technical interviews
- Anyone who wants a simple project to understand JWT and MongoDB
- Learners who want hands-on practice with REST APIs

Project Instruction:

- SpendWise is a backend-only project built using Node.js and Express.

Quick Start:

- npm init
- npm install
- npm run dev

Steps to Create the Project:

- Choose or create a directory for the backend project.
- Open terminal and navigate to that directory.
- Initialize Node.js project using npm init.
- Install required packages: express, mongoose, jsonwebtoken, bcrypt, cors, dotenv.
- Create folders: models, routes, controllers, middleware, config.
- Start the server using npm run dev.
- Test APIs using Postman or Thunder Client at http://localhost:5000.

Pre-requisites :

Software requirements:

- **Node.js**
- **MongoDB**
- **npm**

- **Git**
- **Code Editor**

Knowledge Requirements:

- JavaScript ES6+
- Node.js and Express.js
- MongoDB and Mongoose
- JWT authentication
- Basic REST API concepts

Technical Architecture:

Backend :

- Controllers
- Models
- Routes
- Middleware
- Configuration

Database Architecture (MongoDB):

- Users Collection:
  _id, name, email, password (hashed), createdAt
- Transactions Collection:

_id, userId (FK), title/description, amount/value, createdAt

- Relationship:

Many Transactions belong to One User.

Features:

1. User Authentication : JWT-based registration and login.

2. CRUD Operations :

Create, Read, Update, Delete transactions

3. Protected Routes: Secure APIs using JWT middleware

4. MongoDB Integration. Store and fetch data using Mongoose

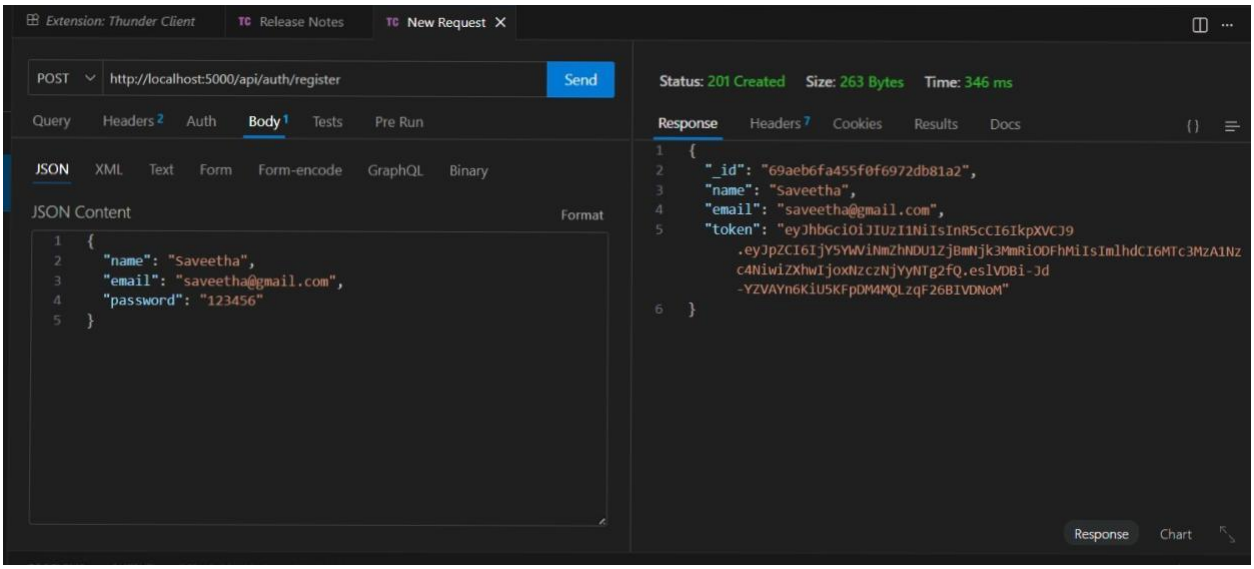
Project Execution :

Project Execution Steps :

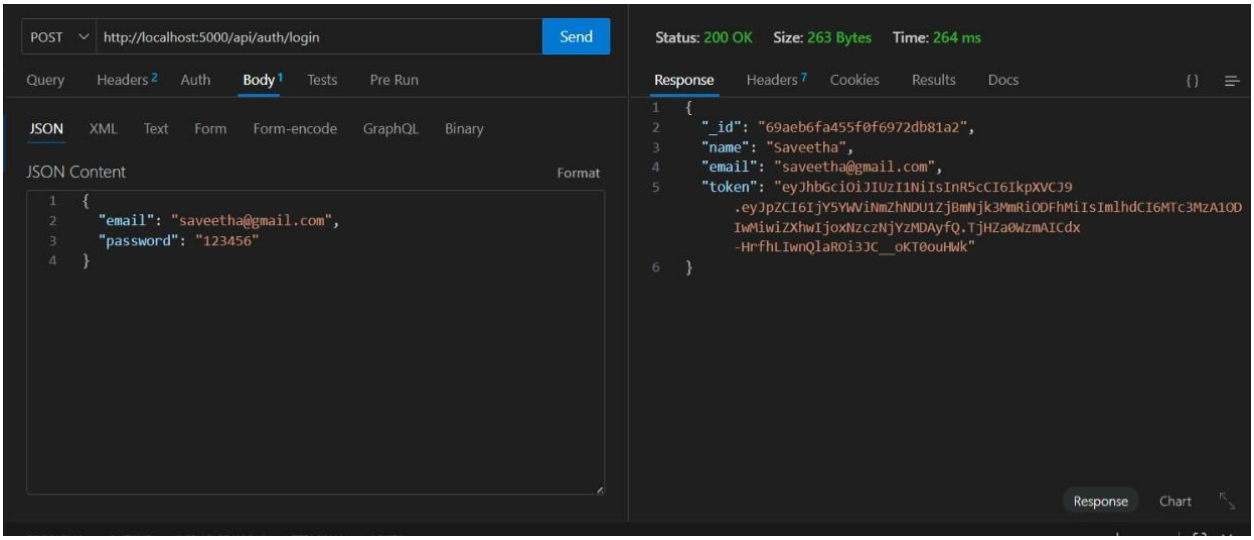
```
C:\spendwise\spendwise>npx nodemon server.js
C:\spendwise\spendwise>npx nodemon server.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙ load multiple .env files with { path: ['.env.local', '.env'] }
Server running on port 5000
MongoDB Connected: ac-sg1nthw-shard-00-02.s3mna8f.mongodb.net
```

OUTPUT SCREENSHOTS :

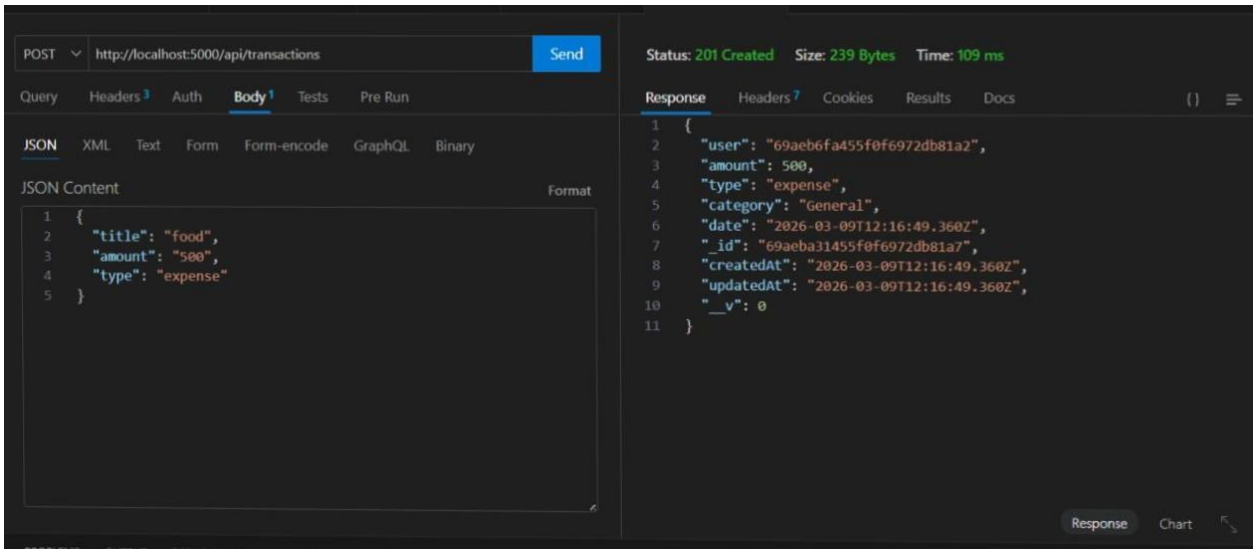
1. Test Register API:



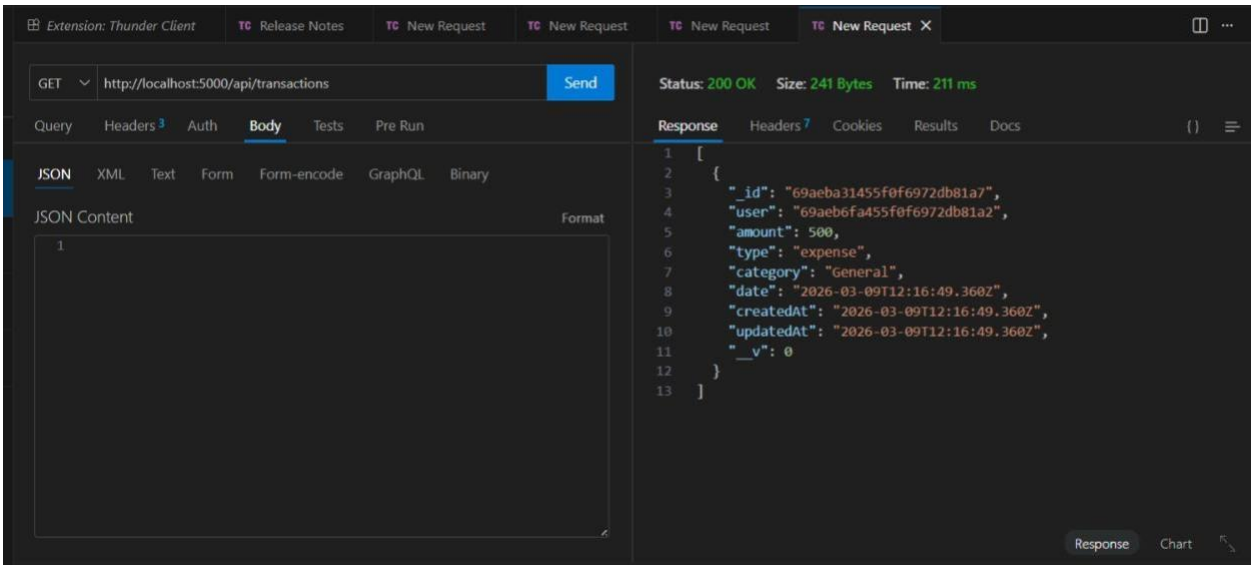
2. Test Login API :



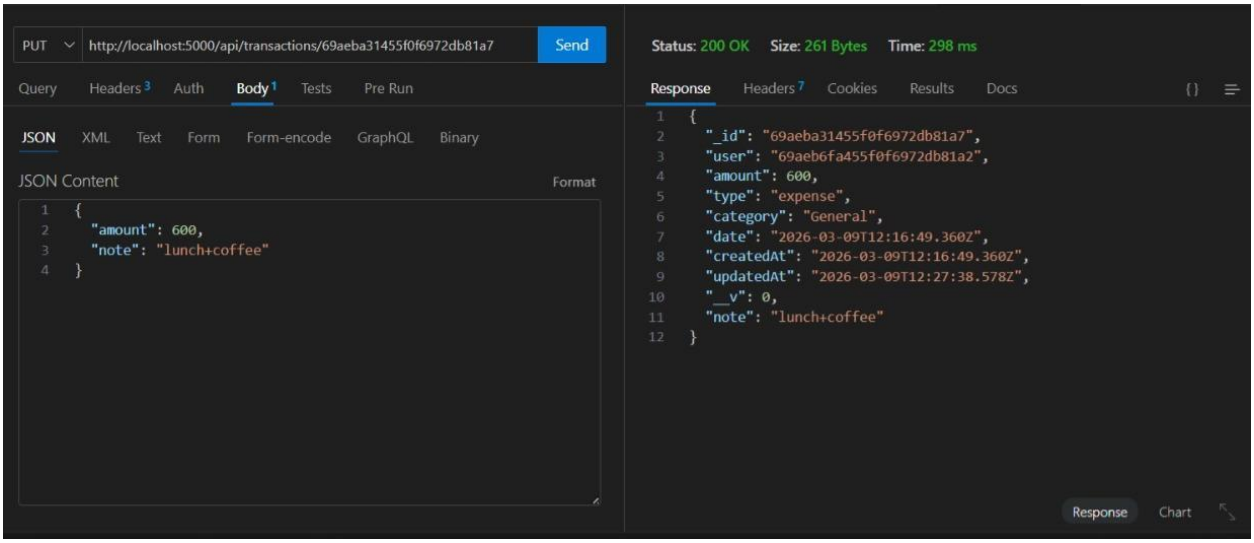
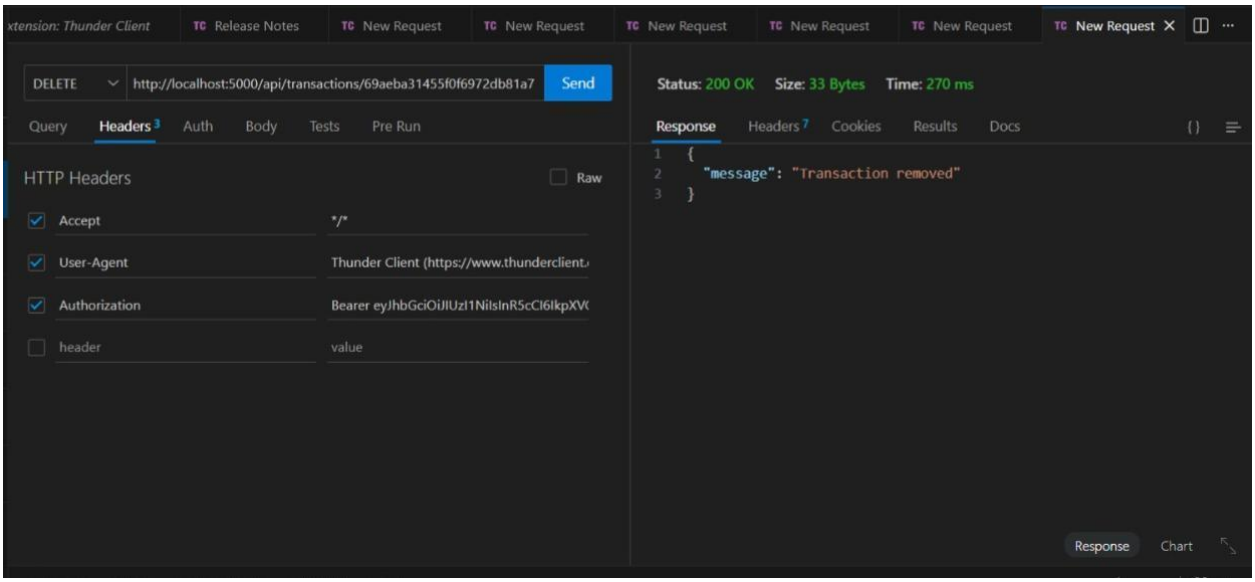
3. Transactions:



4. Get Information :



5. Update API :



6. Delete API :